

hypervisor has to do is schedule guests, manage memory and arbitrate access to hardware?

The Linux kernel has been doing all of this for years. Therefore, the kernel code could easily be leveraged to perform all these tasks. And the addition of code to handle the new CPU instructions and state would make Linux itself function as the hypervisor and host VMs.

The KVM project was started for doing just this, and it was evident by the quick developer acceptance that this really is how virtualisation on Linux was finally going to be acceptable. The KVM project was announced in late 2006 and was accepted the same year in Linus' kernel tree. On the other hand, the Xen Dom0 code has yet to find upstream acceptance. The Xen hypervisor, itself bearing Linux code, will always continue to be a separate project.

The open source advantage

A commonly-cited advantage of open source software is the 'more eyeballs' concept. As more people look at the code, bugs become more obvious and get fixed faster, often before the code enters a stable release. This is definitely true. However, there are other advantages when it comes to open source software with large communities, beyond just more eyeballs.

If one follows the LWN.net "Who develops Linux" articles, it's clear that most of the developers are sponsored to work on Linux by companies. It isn't a big surprise to people any more that companies are running businesses and making profits by relying on open source software. Linux already runs on the widest array of platforms—it can run on simple embedded devices and also on big supercomputers, including everything in between. The developers come from not just one part of the world, but from everywhere. The experience, culture and insights they all bring in are invaluable.

Contrast this to a proprietary OS maker. Perhaps all the developers sit in one campus and are probably used to following a particular train of thought. Just one company cannot match the resources that 50 companies (and, of course, the individuals in the community) put together to collaboratively enhance the OS.

Red Hat, IBM, Novell, Intel, AMD, HP, Fujitsu, Oracle, Nokia and Google, all figure on the latest LWN.net compilation for companies that are funding developers to contribute to the Linux kernel. The sheer scale at which the development happens is mind-boggling.

This, however, does not mean that companies can push whatever code they want into the repositories. Merit wins. There is a peer review of all the patches that flow in. There are people who deeply care about the code that gets accepted. Almost all the patches submitted the first time have to be adjusted after review comments by others. There hardly are patches that go in their unmodified form from the time they were first sent out for review. In many cases, people maintaining subsystems that reject patch submissions could be working for the same company that's promoting the patches. And there's no love lost. Everyone involved understands the prime cause: to create better software.

People understand this, and the companies involved understand this too.

Now why does all this matter in the virtualisation perspective? It's simple. The Linux kernel itself is a hypervisor. Any advances in Linux, the operating system, are directly beneficial to Linux, the hypervisor. By using the KVM technology, guests running on top of KVM can enjoy the benefits immediately when patches get accepted to Linux. KVM guests can already enjoy the support of 64 vCPUs (and more!), huge-page backed memory, a wide range of memory over-commit options, NUMA support and so on. And KVM is just four years old. It has taken other projects many more years to reach the state they currently are in, and even then, they do not offer some of the features that KVM offers. It's an interesting exercise for the reader: compare the feature set as announced in releases of virtualisation software one year back to the current set. The number of features and enhancements KVM can provide in one year's time, others would only dream of achieving in five.

This comes as no surprise. There are two basic mindsets at play. First, the UNIX one: 'Do one thing and do it right'. The KVM developers just focused on providing the best support to exploit the hardware support for virtualisation and left the CPU scheduling, memory management, etc, to Linux. KVM also leveraged the QEMU project heavily that provides a device model. The virtual computer that gets exposed to the guest is provided by QEMU, and KVM developers have heavily updated the upstream QEMU code to enable it to support modern devices, KVM-based guests and a lot of optimisation.

The second philosophy is to contribute as much as possible to upstream software, fighting the urge to ship a forked copy of the codebase with some features that would be deemed controversial upstream, or which would take a longer time to gain acceptance. This might result in some features getting delayed as discussions pan out, and developers pitch in with their opinions on how to do things the 'right' way. But, in the end, the best technical solution wins and maintaining the solution that's accepted by all is easier in the long run. With most enterprise Linux vendors offering seven-year support guarantees, this becomes a big plus. This is because keeping the private functionality in the stable offering working, while also backporting fixes and optimisations from an upstream codebase that changes more and more each day, would soon become a nightmare for the maintainers of the enterprise software.

Just comparing the two open source virtualisation solutions, Xen and KVM, shows us the stark contrast in these principles and the benefits of collaborative development. 

By: Amit Shah

The author is a part of the virtualisation team at Red Hat and is excited to be a part of the technology that's rediscovering commodity x86 servers.